

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 3 – Experiments (5 QUESTIONS)

Course Code:	MLA0407	Course Title:	Deep Learning
CO Assessed:	CO1	Assessment:	Assessment Tool 3 – Experiments
Weightage:	35% of CIA	Total Marks:	(5 Questions × 10 Marks)
CO1 Statement:	Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.		
Student Name:	M.Kousik Reddy	Reg. No.:	192425185
Section / Batch:	2024	Date:	

Code of Conduct

I M.Kousik Reddy certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _kousik reddy

Experiment 1: Implementation of a Single Artificial Neuron

Develop a Python program to simulate an artificial neuron that accepts multiple inputs, computes the weighted sum, applies a step activation function, and generates the output. Test the neuron using different input combinations and analyze its behavior.

Objective

To implement a single artificial neuron using Python and understand the basic working principle of artificial neural networks.

Theory

An artificial neuron is the fundamental building block of an Artificial Neural Network (ANN). It is inspired by the biological neuron present in the human brain. A biological neuron receives signals, processes information, and produces an output. Similarly, an artificial neuron receives input values, processes them mathematically, and generates an output.

An artificial neuron consists of three major components:

1. **Input Values:**

Inputs represent the features provided to the neuron. Multiple inputs can be given to represent different characteristics of data.

2. **Weights:**

Each input is assigned a weight that represents its importance. Higher weights indicate greater influence on the output.

3. **Bias:**

Bias is an additional parameter added to shift the activation function and improve learning capability.

The neuron first calculates the weighted sum:

$$Z = \sum_{i=1}^n x_i w_i + b$$

where:

- x_i = Input values
- w_i = Weights

- b = Bias
- Z = Weighted sum

After calculating the weighted sum, an activation function is applied to produce the final output. The step activation function is commonly used for binary classification.

$$f(Z) = \begin{cases} 1, & Z \geq 0 \\ 0, & Z < 0 \end{cases}$$

Artificial neurons are used in many applications such as image classification, medical diagnosis, speech recognition, and pattern detection.

Task / Procedure

Develop a Python program to simulate the working of a single artificial neuron.

The following steps are performed:

1. Provide multiple input values to the neuron.
2. Assign suitable weights for each input.
3. Initialize a bias value.
4. Calculate the weighted sum of all inputs.
5. Add the bias value to the weighted sum.
6. Apply the step activation function.
7. Generate the final output.
8. Test the neuron using different input combinations.
9. Observe how changes in input values and weights affect the output.
10. Analyze the behavior of the artificial neuron.

Result

The artificial neuron was successfully implemented. The neuron accepted multiple inputs, calculated the weighted sum, applied the step activation function, and generated the required output. The experiment helped in understanding the basic operation of neural network processing units.

Experiment 2: Perceptron for AND Gate

Write a Python program to implement the Perceptron Learning Algorithm for the AND logic gate. Train the model using suitable weights and bias, display the final weights after training, and evaluate the prediction accuracy.

Objective

To implement a perceptron model and understand how a single-layer neural network learns a linearly separable binary classification problem.

Theory

A perceptron is a simple single-layer neural network introduced by Frank Rosenblatt. It is mainly used for binary classification problems. A perceptron receives input values, multiplies them with weights, adds bias, and applies an activation function to produce output.

The mathematical model of a perceptron is:

$$Y = f\left(\sum X_i W_i + b\right)$$

The AND gate is a logical operation where the output is 1 only when all inputs are 1. The truth table of AND gate is:

Input 1	Input 2	Output
0	0	0
0	1	0
1	0	0
1	1	1

The perceptron learning algorithm updates weights based on prediction error.

Weight update rule:

$$W_{new} = W_{old} + \eta(Error)(Input)$$

where:

- η = Learning rate
- Error = Actual output - Predicted output

Since AND gate is linearly separable, it can be solved using a single-layer perceptron.

Task / Procedure

Implement a perceptron learning algorithm for the AND gate.

The following steps are performed:

1. Create the AND gate truth table.
2. Initialize weights and bias values.
3. Calculate the weighted sum.
4. Apply the activation function.
5. Compare predicted output with actual output.
6. Calculate the error value.
7. Update weights and bias using the learning rule.
8. Repeat training until correct classification is achieved.
9. Display the final weights after training.
10. Evaluate the prediction accuracy.

Result

The perceptron successfully learned the AND gate logic. The trained model correctly classified all input combinations and achieved accurate predictions.

Experiment 3: Perceptron for OR Gate

Implement a perceptron to classify the OR logic gate. Compare the training process and final weights with those obtained for the AND gate and analyze the differences.

Objective

To design a perceptron-based binary classification model for the OR gate and study the learning behavior of perceptrons for different logical operations.

Theory

The OR gate is a binary logical operation where the output becomes 1 if at least one input value is 1. Similar to the AND gate, the OR gate is also linearly separable and can be implemented using a single-layer perceptron.

OR gate truth table:

Input 1	Input 2	Output
---------	---------	--------

0	0	0
---	---	---

0	1	1
---	---	---

1	0	1
---	---	---

1	1	1
---	---	---

During training, the perceptron adjusts weights and bias values until it finds a suitable decision boundary that separates the two classes.

The main difference between AND and OR perceptrons is the required weight values and bias because their decision boundaries are different.

Task / Procedure

The OR gate perceptron is implemented using the following steps:

1. Create OR gate input-output combinations.
2. Initialize weights and bias values.
3. Calculate weighted summation.
4. Apply activation function.
5. Compare predicted and actual outputs.
6. Calculate classification error.
7. Update weights and bias.
8. Repeat training until convergence.
9. Display final weights.
10. Compare results with the AND gate perceptron.

Result

The perceptron successfully classified the OR gate outputs. The experiment demonstrated that perceptrons can solve linearly separable logical problems.

Experiment 4: XOR Classification Analysis

Attempt to train a perceptron for the XOR logic gate. Observe the results and explain why the perceptron fails to classify XOR correctly. Suggest an alternative neural network architecture capable of solving this problem.

Objective

To study the limitations of a single-layer perceptron and understand why multilayer neural networks are required for solving non-linear classification problems.

Theory

The XOR (Exclusive OR) gate produces an output of 1 when the input values are different and produces 0 when both inputs are identical.

XOR truth table:

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Unlike AND and OR gates, XOR is not linearly separable. A single-layer perceptron can only create a linear decision boundary, so it cannot classify XOR correctly.

The limitation of perceptrons led to the development of Multilayer Perceptrons (MLP). MLP contains hidden layers that use non-linear activation functions to solve complex problems.

Task / Procedure

The XOR classification problem is analyzed using a perceptron.

Steps performed:

1. Create XOR dataset.
2. Train a single-layer perceptron.
3. Observe prediction results.
4. Identify classification errors.
5. Analyze why the perceptron fails.

6. Study linear and non-linear separability.
7. Suggest MLP architecture as a solution.
8. Explain the role of hidden layers.

Result

The single-layer perceptron failed to classify the XOR problem because XOR is not linearly separable. A multilayer perceptron with hidden layers is required to solve this problem.

Experiment 5: Gradient Descent Optimization

Implement the Gradient Descent algorithm to minimize a simple quadratic cost function. Plot the cost function against the number of iterations and analyze the convergence behavior for different learning rates.

Objective

To understand the working of Gradient Descent optimization and analyze how learning rate affects the convergence and performance of the algorithm.

Theory

Gradient Descent is an optimization technique used to minimize the error or cost function in machine learning models. It is widely used for training neural networks by adjusting weights and reducing prediction error.

The main objective of Gradient Descent is to find the minimum value of a cost function.

The weight update formula is:

$$W_{new} = W - \alpha \frac{dJ}{dW}$$

where:

- W = Weight parameter
- α = Learning rate
- J = Cost function

The learning rate controls how quickly the model learns. A small learning rate provides slow but stable learning, while a large learning rate may cause unstable updates.

Gradient Descent is used in deep learning algorithms such as neural networks, regression models, and optimization problems.

Task / Procedure

Implement Gradient Descent to minimize a quadratic cost function.

Steps performed:

1. Define a quadratic cost function.
2. Initialize parameter values.
3. Calculate gradient values.
4. Update parameters using Gradient Descent.
5. Repeat the process for multiple iterations.
6. Record cost values.
7. Analyze reduction in cost.
8. Test different learning rates.
9. Compare convergence behavior.
10. Identify the suitable learning rate.

Result

The Gradient Descent algorithm successfully minimized the cost function by updating parameters iteratively. The experiment demonstrated how optimization techniques improve the learning process of neural networks.